

# Everything is here: from do to html

Huanyu Jia  
School of Business  
Zhengzhou University  
Zhengzhou, Henan/China  
jiahuanu@zzu.edu.cn

Kerui Du  
School of Management  
Xiamen University  
Xiamen, Fujian/China  
kerrydu@xmu.edu.cn

**Abstract.** This paper introduces `ishere` and `tohtml`, two Stata commands designed to automate the conversion of Stata do-files and log files into Markdown and HTML reports. Unlike existing comprehensive solutions that offer extensive features and polished output, this package deliberately prioritizes simplicity and ease of adoption. The design philosophy, captured in the phrase “everything is here: from do to html” emphasizes minimalist implementation with the lowest possible learning curve. Users mark key locations in their do-files with simple `ishere` commands, run their analyses as usual, and use `tohtml` to transform the log into Markdown and HTML without leaving the Stata environment or adopting new programming paradigms. This package also offers a streamlined approach for AI agents to collect Stata codes and relevant outputs, facilitating efficient report generation.

**Keywords:** st0001, Stata, reproducible research, dynamic documents, Markdown generation, literate programming, workflow integration

## 1 Introduction

Reproducible research has become a cornerstone of modern statistical analysis, requiring seamless integration of code, results, and narrative into comprehensive reports. In the Stata ecosystem, researchers traditionally face challenges when attempting to create publication-ready documents that combine analysis code, results, and interpretive text (Barba 2016).

Several excellent solutions exist within Stata. The built-in `dyndoc` and `dyntext` commands provide sophisticated dynamic document generation with extensive formatting options. Community-contributed packages like `markdoc` (Haghighi 2016a) and `markstat` (Rodríguez 2017) offer powerful features including multiple output formats (PDF, Word, HTML), advanced styling capabilities, and comprehensive markup support. These tools are mature, feature-rich, and capable of producing highly polished, publication-ready documents directly from Stata.

However, these comprehensive solutions necessarily involve learning curves proportional to their feature sets. Users must master specific syntax conventions, understand template systems, or adopt new workflow paradigms. For researchers who need professional output and have time to invest in learning these tools, they represent excellent choices. External solutions like Jupyter notebooks with Stata kernels or RMarkdown with Stata chunks offer even more powerful capabilities but require researchers to work

outside their familiar Stata environment entirely.

This paper introduces a different approach embodied in two commands: `ishere` and `tohtml`. Rather than attempting to replicate the comprehensive feature sets of existing solutions, this package deliberately occupies a different niche: **maximum simplicity with minimum learning investment**. The design philosophy is captured in the phrase “everything is here: from do to html”: “**is here**” refers to the “`ishere`” command, which marks where elements belong in the report; “**to html**” refers to the “`tohtml`” command, which converts marked logs into clean Markdown; “**everything**” emphasizes that all components Stata can provide—code, results, figures, tables—live together in a single do-file

We consciously sacrifice feature comprehensiveness for ease of adoption. The package does not attempt to match the polished output or extensive formatting options such as `markdoc`, `dyndoc` or `markstat`. Instead, it focuses on generating a html file of research report. Meanwhile, it also generates a clean Markdown as a **semi-finished product** that contains all essential elements extracted from Stata. Users can then leverage powerful, user-friendly Markdown editors such as VS Code, Obsidian, or Notion to refine the Markdown into their final document. These editors offer intuitive interfaces, real-time preview, extensive formatting options, and often require no coding knowledge. This division of labor plays to the strengths of each tool: Stata extracts analytical content, specialized editors handle document polishing.

The key innovation lies in maintaining extreme simplicity in the Stata component. Users continue writing standard Stata do-files, adding only occasional `ishere` markers to indicate report structure. There are no new file types to learn, no external dependencies to install, and no fundamental changes to existing workflows. The learning curve is minimal: if you can write a Stata do-file, you can generate results in Markdown and HTML without additional efforts.

The following sections describe the design principles, core functionality, and practical usage of the `ishere/tohtml` package, demonstrating how deliberate simplicity can serve as an entry point to reproducible research workflows while maintaining flexibility through integration with external editing tools.

## 2 Methodology and Features

### 2.1 Design Philosophy

The `ishere/tohtml` package is built on three core principles:

- 1. Zero-Intrusion Design:** Users maintain their standard Stata workflow without adopting new programming paradigms. The commands act as auxiliary markers that do not interfere with data manipulation or statistical analysis. A do-file with `ishere` markers remains a valid, executable Stata script that produces identical statistical results whether or not the markers are processed.

**2. Minimal Learning Investment:** The entire command set can be learned in minutes. Basic usage requires understanding only two commands (`ishere` and `tohtml`) with intuitive syntax. Advanced features remain optional and can be adopted incrementally as needs arise.

**3. Single-Source Workflow:** All components of the report—code, results, figures, tables, and structural markers—reside in a single do-file. There are no separate template files to maintain, no external configuration files to manage, and no risk of documentation becoming desynchronized from analysis. This workflow essentially serves as an extended built-in logging function integrated with Markdown capabilities.

## 2.2 The `ishere` Command: Dynamic Placeholders

The name `ishere` draws inspiration from a long-standing convention in academic and technical writing: placeholder notes such as “Table 1 inserts here” or “Figure 1 is here”. For decades, researchers have used such phrases to indicate where results should appear in a manuscript, especially when figures and tables are produced separately from the main text.

In reproducible research with Stata, `ishere` transforms this passive convention into an active and executable placeholder system. Conceptually, it means “insert something here, on this line, in this do-file”. Rather than merely annotating the script for human readers, `ishere` creates structural markers or embedded content that are later interpreted by `tohtml` when constructing the final report.

### Basic Syntax

The command operates in three distinct modes: *placeholder mode*, *insertion mode*, and *dynamic-text mode*. Together, these modes allow `ishere` to bridge analytical code, narrative explanation, figures, tables, and computed results within a single reproducible workflow.

#### *Placeholder mode*

In *placeholder mode*, `ishere` produces no substantive visible output during execution. Instead, it leaves structural markers in the log file that `tohtml` later parses and converts into formatted sections of the final report. These markers act as instructions for organizing code blocks, headings, text blocks, figures, and tables.

```
. // Mark code block boundary
. ishere ```
. // Alternative code block marker
. **```

. // Insert markdown heading
. ishere # Main Title
. // Alternative markdown heading
. **# Main Title
```

```
. // Insert subheading
. ishere ## Subtitle
. // Alternative subheading
. ##### Subtitle

. // Begin text block
. ishere /*
. // Alternative text block beginning
. **/*

. // End text block
. ishere */
. // Alternative text block ending
. ***/
```

In this sense, `ishere` functions as a structural annotation device. It does not itself produce the final document, but it defines where and how elements should appear once the log is post-processed.

#### *Insertion mode*

In *insertion mode*, `ishere` actively emits Markdown or HTML code into the log file. This makes it possible to insert figures and tables directly into the eventual report rather than merely marking their locations. Compared with placeholder mode, this mode is more explicit and generative: the relevant embedding code is written into the log at the moment of execution.

```
. // Insert figure with default settings
. ishere fig using "scatter.png"


. // Insert figure with zoom control
. ishere fig using "scatter.png", zoom(80%)


. // Insert figure with custom dimensions
. ishere fig using "scatter.png", height(400px) width(600px)


. // Insert HTML table
. ishere tab using "results.html"
<iframe src='results.html' width='100%' height='400px' frameborder='0'></iframe>

. // Insert table from Markdown file
. ishere tab using "results.md"
<iframe results.md ></iframe>

. // Insert table with custom iframe dimensions
. ishere tab using "results.html", height(400px) width(600px)
<iframe src='results.html' width='600px' height='400px' frameborder='0'></iframe>
```

This mode supports various image formats (PNG, JPG, JPEG, SVG, GIF, BMP, WEBP) as well as tables stored in either HTML or Markdown format. HTML/Markdown tables can be embedded via iframes. It is therefore suitable when analytical outputs have already been exported and need to be inserted directly into the report in a portable and reproducible manner.

### *Dynamic-text mode*

In *dynamic-text mode*, **ishere** supports the insertion of computed values into narrative text through the **ishere display** syntax. This functionality is designed for a common reporting need: incorporating quantities such as sample sizes, means, fit statistics, or regression coefficients into prose without manual copy-paste. In this way, the explanatory text automatically remains synchronized with the latest analytical results.

```
* compute a value
regress price mpg weight
local r2 = e(r2)

* emit the value to the log
ishere display %5.3f `r2'

* write narrative text with a placeholder
ishere /*
* The model R-squared is {ishere display %5.3f `r2'}.
ishere */
```

When **tohtml** processes the log, it replaces the placeholder `{ishere display ...}` inside the text block with the corresponding displayed value. As a result, narrative text can be updated automatically whenever the analysis is rerun.

Each **ishere display** applies only to the **first** text block (`ishere /*...ishere */`) or (`/**/...**/`) that appears **after** it in the log. If the same value is needed in multiple text blocks, the user should issue another **ishere display** command before each relevant block. This design keeps the mapping between computed values and narrative placeholders explicit and predictable. Note that the content inside the text block complies with Markdown syntax. In this scenario, the asterisk (\*) functions as a list marker rather than a comment symbol. The mathematical equations can also be added in the text environment.

Taken together, these three modes make **ishere** the core placeholder and insertion mechanism of the package. It provides the link between code execution and document structure, allowing Stata scripts to function not only as analytical programs but also as reproducible and readable research records.

## 2.3 The **tohtml** Command: Log-to-HTML Conversion Engine

The **tohtml** command is specifically designed to work hand-in-hand with **ishere**, forming a unified workflow embodied by the phrase *everything is here: from do to html*.

This design philosophy emphasizes a minimalist approach to reproducible research, in which “is here” corresponds to the `ishere` command for marking where elements belong, and “to html” corresponds to the `tohtml` command for converting marked outputs into polished HTML reports.

### Basic Syntax

The `tohtml` command operates in two distinct modes: *log-file mode* and *directory mode*.

#### *Log-file mode*

```
tohtml {path_to_logfile} [ , cleanmd(string) html(string) replace css(string)
    rpath(string) wpath(string) clean cleancode(path_to_dofile) ]
```

In *log-file mode*, `tohtml` takes a Stata log file containing `ishere` markers as input, cleans the log, parses its structural markers, and optionally converts the resulting Markdown into an HTML report. This mode is intended for reproducible reporting directly from analytical logs.

More specifically, log-file mode provides several key features. First, it performs intelligent parsing of log content, distinguishing between code, output, headings, and embedded elements such as figures and tables. Second, it supports multiple output modes, allowing users to generate either a detailed report that preserves all code and output, a minimal report that retains only headings and selected embedded content, or a merged report that replaces log-recorded code with cleaner source code from the original do-file. Third, it supports professional HTML styling through either built-in GitHub-style CSS or user-supplied stylesheets; when the GitHub-style option is used, MathJax support is also enabled for rendering mathematical expressions. Fourth, it provides flexible path management through resource path replacement, which is useful for producing portable reports across different working environments.

Within log-file mode, `tohtml` supports three output variants. The *standard mode*, which is the default, preserves all code and output and thus produces comprehensive documentation of the analytical session. The *clean mode*, invoked by the `clean` option, retains only headings, figures, tables, and text blocks while removing code and console output. The *cleancode mode*, invoked by the `cleancode(dofile)` option, displays clean source code from the original do-file while preserving execution results from the log. Together, these alternatives make log-file mode suitable for different documentation needs, ranging from full reproducibility records to concise presentation-oriented reports.

#### *Directory mode*

```
tohtml {path_to_directory} [ {path_to_directory} ... ] [ , html(string) replace
    css(string) width(string) height(string) zoom(string) ]
```

In *directory mode*, `tohtml` takes one or more directories as input and automatically gathers exported figure files and HTML/Markdown table files from those directories, combining them into a single HTML report. This mode is designed for batch integration of outputs that have already been produced and stored across folders. It is particularly useful when figures and tables are generated in different subdirectories but need to be assembled into a unified HTML document. This capability extends the command beyond simple log conversion and makes it suitable for modular reporting workflows.

Taken together, the two modes make `tohtml` a flexible log-to-HTML conversion engine. The first mode emphasizes reproducible transformation from annotated logs to reports, whereas the second emphasizes batch aggregation of pre-exported analytical outputs.

## Options

Because `tohtml` supports two modes of operation, some options apply primarily to log-file processing, whereas others are effective only in directory processing.

`cleanmd(filename)` specifies the output path for the cleaned Markdown file. If this option is not specified, `_clean` is appended to the input filename by default. This option is used in *log-file mode*.

`html(filename)` converts the cleaned Markdown file to HTML after processing in *log-file mode*, and specifies the name of the combined HTML report in *directory mode*. This option requires the `markdown` command when HTML is generated from cleaned Markdown.

`replace` overwrites existing output files. If this option is omitted and the target file already exists, the command returns an error. This option can be used in both modes.

`css(filename|githubstyle)` applies CSS styling to the generated HTML file. Users may either specify the path to a custom CSS file or use `githubstyle` to invoke the built-in GitHub-style stylesheet. This option requires `html()`. The CSS file is copied to a `css` subdirectory in the location of the HTML output. This option can be used in both modes.

`rpath(directory)` specifies the local path of resource files, such as images and tables. This path is replaced by the path specified in `wpath()`, making it useful for generating portable reports. This option is used in *log-file mode*.

`wpath(url)` specifies the target path for resource files, typically a relative path such as `"/"` or a URL. In the generated Markdown or HTML output, `rpath()` is replaced by this path. This option is used in *log-file mode*.

`clean` activates the minimal cleaning mode, keeping only headings beginning with `#`, `<img>` tags, `<iframe>` tags, and content within `ishere` text blocks, while removing all code and output. This option is used in *log-file mode*.

`cleancode(dofile)` activates the code-merging mode. It uses the original code from the specified do-file to replace code captured in the log file while preserving the execution results from the log. The do-file must contain `ishere` markers indicating the positions of figures, tables, and related elements. This option is suitable when users want to display clean source code from the do-file alongside execution results from the log in the final report. This option is used in *log-file mode*.

`width(string)` sets the default width for all images and tables. This option is effective only when processing directories and is therefore specific to *directory mode*.

`height(string)` sets the default height for all images and tables. This option is effective only when processing directories and is therefore specific to *directory mode*.

`zoom(string)` sets the default zoom level for all images. This option is effective only when processing directories and is therefore specific to *directory mode*.

## 2.4 Workflow Integration

The typical workflow follows these steps:

1. Write a standard Stata do-file with `ishere` markers
2. Run the do-file with logging enabled (`log using`)
3. Process with `tohtml` to generate HTML

This workflow requires no specialized editors, no external compilation tools, and no changes to the underlying statistical analysis. The report generation happens as a post-processing step, completely separate from the analysis itself.

## 2.5 Comparison with Alternative Solutions

This section acknowledges the strengths of existing solutions while clarifying our package's distinct positioning.

**vs. dyndoc/dyntext:** Stata's built-in dynamic document commands are powerful, mature solutions offering sophisticated templating, conditional content, and direct generation of multiple output formats. They produce polished documents with professional styling out-of-the-box. However, they require maintaining separate template files (`.dyndoc`, `.dyntext`) and learning specialized syntax including tag structures and template logic.

Our `ishere/tohtml` approach keeps everything in the do-file using minimal marker syntax, trading the comprehensive features of `dyndoc/dyntext` for simplicity and faster adoption. Our output (clean Markdown) is deliberately a semi-finished product, suitable for users who prefer to do final formatting in more intuitive Markdown editors.



**vs. `stmd` (Hemken 2019):** `stmd` is a useful bridge between Stata’s built-in dynamic-document system and standard Markdown writing. Rather than requiring users to work directly in Stata’s native `dyndoc` syntax, it allows them to write dynamic documents in more conventional Markdown style and then converts these documents to HTML through `dyndoc`. This makes `stmd` particularly attractive for users who appreciate the capabilities of `dyndoc` but prefer a simpler and more familiar Markdown-based writing experience. In that sense, it offers a practical compromise between the power of Stata’s built-in dynamic-document tools and the readability of standard Markdown source files.

However, `stmd` still remains firmly within the dynamic-document paradigm: users must create and maintain a dedicated document source file written in dynamic Markdown, and document generation continues to rely on the `dyndoc` workflow underneath. By contrast, the `ishere/tohtml` approach is intentionally less demanding. It does not ask users to write or maintain a separate document file, nor does it require them to treat the source as a Markdown authoring environment. Instead, users continue to work in ordinary Stata do-files, add only lightweight marker commands, and then convert the resulting log into clean Markdown and HTML through post-processing. We therefore give up the tighter integration with `dyndoc` and the more formal dynamic-Markdown workflow of `stmd` in exchange for lower entry costs, less source-file overhead, and a reporting process that stays much closer to everyday Stata practice

**vs. `markdoc` (Haghighi 2016a, 2020):** `markdoc` is an excellent, feature-rich package offering multiple output formats (HTML, PDF, LaTeX, Word, EPUB), extensive styling options, Markdown dialect support, and sophisticated document structure control. It can produce publication-ready documents with beautiful typography and complex layouts. These capabilities make it ideal for users seeking comprehensive, end-to-end solutions.

However, `markdoc` requires writing Markdown syntax directly in do-file comments using special comment delimiters and adopting a specific workflow paradigm where the do-file becomes a literate programming document. Our `ishere/tohtml` approach uses simple marker commands (`ishere # Title`, `ishere fig using...`) that feel more like standard Stata commands, preserving the do-file’s primary identity as analysis code rather than document source. We intentionally provide fewer features but dramatically reduce the learning curve.

**vs. `markstat` (Rodríguez 2017):** `markstat` implements the classic “literate programming” approach championed by Knuth (1992) and extended to Stata by (Haghighi 2016b), rodriguez2017. It uses a single `.stmd` file that mixes Markdown narrative and Stata code, with simple indentation rules (one tab or four spaces) to distinguish code blocks. `markstat` represents a comprehensive, end-to-end solution that produces publication-ready documents. It is ideal for users who need professional output immediately and are comfortable adopting Markdown-based literate programming workflows. The approach trades a higher initial learning curve for maximum feature completeness and direct-to-publication capability.

While `markstat` follows the literate programming paradigm of mixing code and documentation in a unified file, our `ishere/tohtml` approach maintains a clear separation

of concerns. Users work entirely in familiar Stata do-files, using `ishere` only as non-intrusive markers, with `tohtml` handling document generation as a post-processing step. This preserves Stata's identity as analysis code rather than document source, appealing to users who prefer to continue their existing workflows with minimal disruption.

Notably, we also develop the `tomarkstat` command<sup>1</sup>, further enhancing compatibility between the two approaches: it allows the Markdown output generated by `tohtml` to be converted into an `.stmd` file, thereby enabling subsequent processing with `markstat` when users require more sophisticated formatting and richer document features. In this sense, our workflow not only offers a lightweight and low-intrusion alternative to `markstat`, but also provides a bridge to its more advanced publishing capabilities.

**vs. `texdoc/webdoc/sttex` (Jann 2016, 2017, 2022)** `texdoc`, `webdoc`, and `sttex` form a coherent family of dynamic-document commands developed by Ben Jann. Although they target different output environments, LaTeX in `texdoc`, HTML/Markdown in `webdoc`, and more efficiently managed LaTeX workflows in `sttex`, they share a common philosophy: the source file also serves as a document-authoring environment in which Stata code is woven into a larger manuscript or report. Across this family, the strengths are clear: close integration of analysis and writing, strong support for structured and reproducible documents, and, especially in the case of `sttex`, improved efficiency for larger projects through selective re-execution of code blocks and section-based dependency management. These commands are therefore particularly attractive to users who are comfortable adopting a full dynamic-document workflow and who want tighter control over the production of LaTeX- or web-oriented outputs.

By contrast, the `ishere/tohtml` approach is deliberately narrower and lighter-weight. Rather than turning the source file itself into a document-production environment, it preserves the ordinary Stata do-file as the primary unit of work and uses only minimal marker commands to identify key locations in the analysis. Document generation is then handled as a post-processing step from the log, yielding clean Markdown and HTML without requiring users to embed substantial LaTeX or HTML/Markdown content in the source file or adopt a more specialized authoring paradigm. In this sense, we intentionally give up the broader feature sets, richer document-structure control, and greater publishing sophistication offered by `texdoc`, `webdoc`, and `sttex` in exchange for a substantially lower learning curve, less disruption to established Stata habits, and a workflow that remains much closer to standard Stata practice.

**vs. `log2markup` (Bruun 2016):** `log2markup` transforms a Stata text log into a markup-based document and can target different markup languages such as Markdown, HTML, or LaTeX. It is attractive for users who prefer a log-centered workflow: analyses are run in the usual way, recorded in a text log, and then converted through a separate transformation step. This makes `log2markup` useful for users who want to build documents from existing Stata logs and are comfortable with a markup-oriented workflow.

However, `log2markup` still relies mainly on post hoc parsing of the log file itself,

---

1. `tomarkstat` can be accessed via `ssc install tomarkstat`

which can make structure recovery less predictable in edge cases. It also requires users to write explicit markup blocks and special prefixes to distinguish text, code, hidden preparation steps, and raw output. By contrast, the `ishere/tohtml` approach combines log-based post-processing with lightweight structural cues written directly in the do-file. Most structural elements are marked with simple `**` syntax, while figures, tables, and dynamic inline values are handled through explicit `ishere` commands. We therefore give up some of the generality of `log2markup` in exchange for cleaner structure recovery and a workflow closer to standard Stata practice.

**vs. `log2html` (Baum et al. 2001):** `log2html` is a compact utility for translating a Stata SMCL log directly into an HTML page. It is especially useful when the main goal is simply to export the contents of the Stata results window into a browser-readable format. It also offers straightforward control over presentation features such as page title, colors, line width, and CSS.

However, `log2html` is primarily a log-translation tool rather than a lightweight document-structuring system. If users want headings, clearer separation of code and narrative, controlled placement of figures and tables, or dynamic values inserted into prose, they typically need to write additional HTML manually. By contrast, the `ishere/tohtml` approach is designed around document structure from the outset: users mark headings, text blocks, figures, tables, and dynamic values in the do-file, and these are reconstructed into clean Markdown and HTML through post-processing. In this sense, `log2html` is best understood as a convenient log-to-HTML exporter, whereas `ishere/tohtml` provides a lightweight reporting workflow.

### 3 Step-by-Step Usage Examples

This section demonstrates practical usage progressing from basic to advanced applications.

#### 3.1 Basic Example: First Report in Five Minutes

The simplest possible workflow creates a report with code, output, and basic structure:

```
* Start logging
log using "example1.log", text replace

* Add a title
ishere # My First Data Analysis

* Mark code block
ishere

* Perform standard analysis
sysuse auto, clear
summarize price mpg weight
regress price mpg weight

* Close log
```

```
log close

* Convert to HTML
tohtml example1.log, html(example1.html) css(githubstyle) replace
```

This example demonstrates the minimal learning investment required: add one `ishere` command for the title, one for the code block, and run `tohtml` for conversion. The entire workflow can be learned in minutes. In this case, `texttttohtml` would intelligently add the missing code block marks to let the titles (subtitles), text blocks, inserting tables and figures outside the code blocks.

## 3.2 Structured Report with Multiple Sections

Building on the basic example, add section headings to organize content:

```
log using "example2.log", replace text

*** Automobile Data Analysis
**** Data Description
****
sysuse auto, clear
describe

*** Price Analysis
****
summarize price, detail
histogram price, frequency

*** Regression Analysis
****
regress price mpg weight foreign

log close
tohtml example2.log, html(example2.html) css(githubstyle) replace
```

This example shows how `ishere` markers create document structure without interfering with the analysis code. The statistical commands remain unchanged from what you would normally write. In this example, we use the alternative marker `**` for the same functions of `ishere` in emitting titles and code block markers.

## 3.3 Dynamic narrative text with `ishere` display

The following example demonstrates how to write narrative text that automatically includes computed values:

```
log using "example3.log", replace text

*** Narrative Example
****
sysuse auto, clear
regress price mpg weight
local r2 = e(r2)
```

```

* emit the value
ishere display %5.3f `r2'

* narrative text
**/*
* The model R-squared is {ishere display %5.3f `r2'}.
***/

log close
tohtml example3.log, html(example3.html) css(githubstyle) replace

```

This pattern avoids manual copying of results into prose and supports formatted numeric output via Stata's display formats (e.g., %5.3f).

### 3.4 Embedding Figures and Tables

For professional reports, embed figures and HTML/Markdown tables directly:

```

log using "example4.log", replace text

*** Complete Analysis Report
***# Data Overview
**`
sysuse auto, clear
summarize

***# Price Distribution
**`
scatter price mpg
graph export "scatter.png", replace
ishere fig using "scatter.png", zoom(80%)

***# Regression Results
**`
regress price mpg weight foreign
* Assume outreg2e or similar command exports to HTML
outreg2e using "regression_table.html", replace html
ishere tab using "regression_table.html",height(700px) width(900px)

log close
tohtml example4.log, html(example4.html) css(githubstyle) replace text

```

Note how figures and tables are inserted at their logical positions in the narrative using `ishere fig` and `ishere tab`. The syntax is intuitive and requires no special knowledge of HTML or markdown. We extend two community-contributed Stata commands, `logout` (Wada 2009) and `outreg2` (Wada 2005), to create enhanced versions `logoute` and `outreg2e`, which are tailored for exporting regression results as Markdown and HTML tables. Both newly developed commands are readily available for installation via the SSC archive. For alternative needs, Stata's built-in `table` command also supports direct export to Markdown and HTML formats.

### 3.5 Clean Mode for Presentation Reports

For reports emphasizing results over console output, use clean mode:

```
log using "example5.log", replace text

*** Analysis Summary
**/*
* This report presents key findings from our automobile data analysis.
* We focus on the relationship between price, fuel efficiency, and weight.
***/

****# Key Statistics
****`
sysuse auto, clear
logoute, save("mytable") replace html : tabstat price mpg weight, s(mean sd min max)

****# Price vs. Efficiency
****`
scatter price mpg
graph export "scatter.png", replace
ishere fig using "scatter.png", zoom(75%)

****# Main Results
****`
regress price mpg weight
outreg2e using "main.results", replace md
ishere tab using "main.results.md"

log close
tohtml example5.log, clean html(example5.html) css(githubstyle) replace text
```

The `clean` option removes all console output and code blocks, keeping only headings, narrative text (from `ishere /* ... ishere */` blocks or `***/* ... ***/` blocks), figures, and tables. This mode is ideal for client reports or presentations where the focus is on results rather than technical details.

### 3.6 Cleancode Mode for Technical Documentation

For documentation that combines clean code with execution results:

```
cap log close
log using "example6.log", replace text

*** Analysis Summary
**/*
* This report presents key findings from our automobile data analysis.
* We focus on the relationship between price, fuel efficiency, and weight.
***/

****# Price vs. Efficiency
****`
sysuse auto, clear
scatter price mpg
graph export "scatter.png", replace
ishere fig using "scatter.png", zoom(75%)
```

```

#### Main Results
****
regress price mpg weight
outreg2e using "main_results", replace html
ishere tab using "main_results.html"

log close
tohtml example6.log, cleancode(example6.do) html(example6.html) css(githubstyle) replace

```

The `cleancode()` option uses the original do-file's clean code while displaying the execution results from the log. This mode is perfect for teaching materials or technical documentation where code readability matters. Figure 1 presents the generated output file `example6.html`. More examples can be accessed on [https://kerrydu.github.io/Stata\\_tohtml/](https://kerrydu.github.io/Stata_tohtml/).

### 3.7 Processing Entire Directories

For batch processing multiple outputs, point `tohtml` at one or more directories:

```

* Generate multiple outputs to a directory
cap mkdir output
cap mkdir output\figures
cap mkdir output\tables

****
sysuse auto, clear
scatter price mpg
graph export "output/figures/figure1.png", replace
scatter price weight
graph export "output/figures/figure2.png", replace

* Process one directory
tohtml "output/figures", html(combined_report1.html) replace

```

The `tohtml` command processes all files in the "output/figures" directory specifically those prefixed with "figure" or "table" and appended with valid supported suffixes. It automatically collects the images and compiles them into a single HTML document.

```

* Export tables to output directory...
****
regress price mpg weight
outreg2e using "output/tables/table1", replace html
ishere tab using "output/tables/table1.html"

regress price mpg length
outreg2e using "output/tables/table2", replace html
ishere tab using "output/tables/table2.html"

* Process two directories
tohtml "output/figures" "output/tables" ,html(combined_report2.html) replace

```

The `tohtml` command automatically collects all files starting with "table" (HTML/Markdown) and "figure" (images) in the two directories, creating a consolidated report. This mode is useful for large-scale analyses generating many outputs.

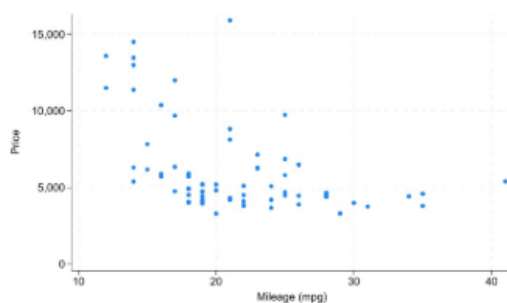
```
log using "example6.log", replace
```

## Analysis Summary

- This report presents key findings from our automobile data analysis.
- We focus on the relationship between price, fuel efficiency, and weight.

## Price vs. Efficiency

```
sysuse auto
scatter price mpg
graph export "scatter.jpg", replace
```



## Main Results

```
regress price mpg weight
outreg2e using "main_results", replace html
```

| VARIABLES    | (1)                 |
|--------------|---------------------|
|              | price               |
| mpg          | -49.51<br>(86.16)   |
| weight       | 1.747***<br>(0.641) |
| Constant     | 1,946<br>(3,597)    |
| Observations | 74                  |
| R-squared    | 0.293               |

Standard errors in parentheses

\*\*\* p<0.01, \*\* p<0.05, \* p<0.1

Figure 1: Screenshot of example6.html



## 4 Technical Implementation

### 4.1 Parsing Strategy

The `tohtml` command employs a multi-stage parsing strategy:

1. **Line-by-line reading** of the translated log file
2. **Prefix removal** of Stata control characters (`{txt}`, `{res}`, `{com}`, etc.)
3. **Marker recognition** to identify `ishere` commands and their types
4. **Code block inference** to determine where code sections begin and end
5. **HTML/markdown merging** for multi-line embedded elements
6. **Path normalization** for cross-platform compatibility

The parser is implemented in Mata for performance and uses vectorized operations wherever possible to handle large log files efficiently.

### 4.2 CSS and MathJax Integration

When using the `css(githubstyle)` option, `tohtml` automatically:

- Generates GitHub-style CSS for professional appearance
- Injects MathJax library for LaTeX mathematical notation support
- Creates responsive design that works across devices
- Provides syntax highlighting for code blocks

Users can also provide custom CSS files for organizational branding or specific formatting requirements.

### 4.3 File Format Support

**Input formats:** - Plain text log files (generated with `log using filename, text`)

**Output formats:** - Cleaned Markdown (.md); HTML with embedded CSS and JavaScript (.html)

**Embedded element formats:** - Images: PNG, JPG, JPEG, SVG, GIF, BMP, WEBP; Tables: HTML/Markdown files (embedded via `iframe`)

## 5 Conclusion

The `ishere` and `tohtml` commands represent a deliberate exercise in minimalist design for statistical computing tools. Rather than competing with comprehensive solutions like `markdoc`, `dyndoc`, or external notebook systems on feature counts, this package occupies a distinct niche: **the simplest possible entry point to reproducible research workflows in Stata.**

We acknowledge that existing solutions (`markdoc`, `markstat`, `dyndoc`, `dyntext`) are more feature-complete and can produce more polished output directly. These are excellent tools for users who need their advanced capabilities and have time to master them. Our package makes a conscious trade-off: we sacrifice feature comprehensiveness for radical simplicity. This approach respects a fundamental reality: many users find comprehensive tools overwhelming and never adopt them, while simple tools that solve 80% of needs see widespread use. We target the “quick start, iterate later” segment of users.

The package succeeds if it enables users to take their first steps toward reproducible research workflows without overwhelming them. By acknowledging that we’re not trying to do everything, just the Stata-specific parts well, we create space for users to leverage the broader ecosystem of excellent Markdown tools. This humility in scope is a feature, not a limitation. Beyond traditional user-driven workflows, the `ishere/tohtml` package offers a natural interface for AI-assisted statistical computing. In this setting, an AI agent generates Stata code, inserts structural markers to delineate sections, code blocks, figures, and tables, and executes the do-file with logging enabled. The agent then calls `tohtml` non-interactively to produce a clean Markdown or HTML report. The resulting HTML file contains all analytical components in a structured, browser-readable format, making it an ideal output medium for automated reporting pipelines where the end user expects a polished summary rather than raw log output. The simplicity of the marker syntax means that AI agents can reliably produce valid `ishere` annotations without extensive prompt engineering or specialized training. We also create an AI agent skill which can be accessed on <https://github.com/kerry4ai/statareport>.

## 6 References

- Barba, L. A. 2016. The hard road to reproducibility. *Science* 354: 142.
- Baum, C. F., N. J. Cox, and B. Rising. 2001. *LOG2HTML: Stata module to produce HTML log files*. Statistical Software Components S422801, revised 05 Nov 2023. <https://ideas.repec.org/c/boc/bocode/s422801.html>.
- Bruun, N. H. 2016. `log2markup`: Stata module to transform a Stata text log into a markup document. *Statistical Software Components* (S458147).
- Haghighi, E. F. 2016a. Markdoc: Literate programming in Stata. *The Stata Journal* 16(4): 964–988.
- . 2016b. Rethinking literate programming in statistics. *The Stata Journal* 16(4): 938–963.
- . 2020. Software documentation with markdoc 5.0. *The Stata Journal* 20(2): 336–362.
- Hemken, D. 2019. `STMD`: Stata module to convert dynamic Markdown to HTML format, using Stata `dyndoc`. *Statistical Software Components* (S458606).

- Jann, B. 2016. Creating LaTeX documents from within Stata using texdoc. *The Stata Journal* 16(2): 245–263.
- . 2017. Creating HTML or Markdown Documents from within Stata using webdoc. *The Stata Journal* 17(1): 3–38.
- . 2022. STTEX: Stata module to integrate Stata results into a LaTeX document. *Statistical Software Components* (S459130).
- Knuth, D. E. 1992. *Literate programming*. CSLI Lecture Notes.
- Rodríguez, G. 2017. Literate data analysis with Stata and Markdown. *The Stata Journal* 17(3): 600–618.
- Wada, R. 2005. OUTREG2: Stata module to arrange regression outputs into an illustrative table. *Statistical Software Components* (S456416). Revised 17 Aug 2014.
- . 2009. LOGOUT: Stata module to convert log or ASCII files into various output formats. *Statistical Software Components* (S457034). Revised 06 Nov 2009.

**About the authors**

Huanyu Jia is an assistant professor at the School of Business, Zhengzhou University, China. Her primary research interests include energy economics and behavioral economics.

Kerui Du (corresponding author) is a professor at the School of Management, Xiamen University, China. His primary research interests include applied econometrics, energy economics, and environmental economics.